

# 목차

- RNN
- LSTM

# RNN (순환신경망)

## Recurrent Neural Network

RNN은 Recurrent Neural Network의  
약자로

시계열 데이터와 같은 연속적인 정보를 처리  
할 때 많이 사용되는 신경망입니다

---

# 시계열성 데이터란?

: 시간의 흐름에 따라 순차적으로 관측되고 기록되는 데이터.  
주로 과거 패턴을 분석하여 미래를 예측하는데 많이 사용됨.

1. 금융 데이터 : 주식가격, 환율, 경제 지표 등 시간이 지남에 따라 변하는 금융데이터
  2. 기상데이터 : 일별, 시간별 온도, 습도, 강수량 등의 기상데이터
  3. 언어데이터 : 자연어 처리(NLP) 작업 - 자동완성, 언어번역 등
  4. 음성데이터 : 음성 인식, 음성 합성 등
-

---

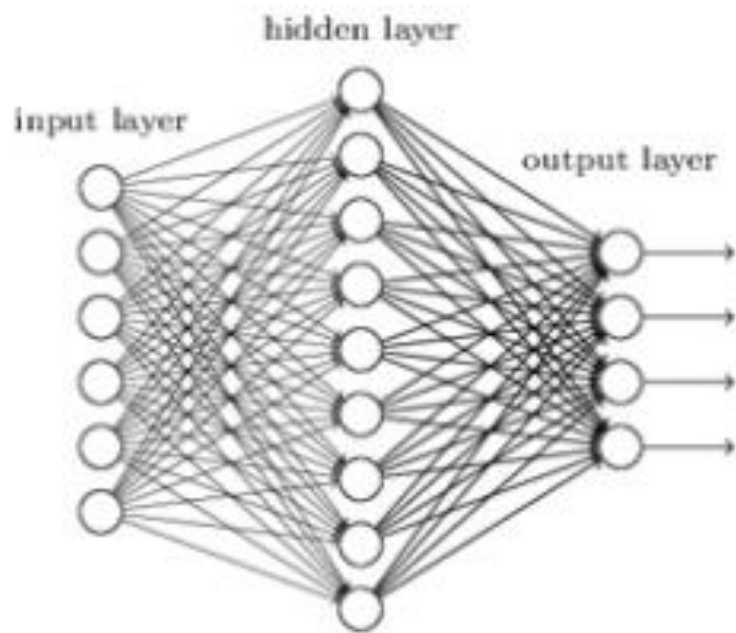
# RNN의 특징

RNN은 기존 신경망과는 다르게 '기억력' 을 가지고 있음.

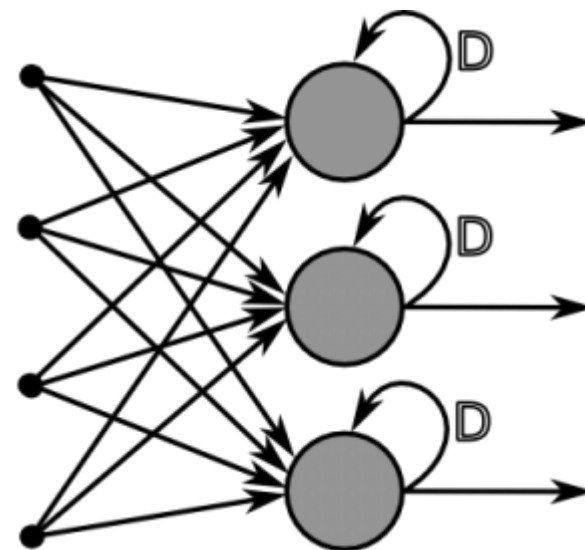
이전에 본 데이터를 기억하고 그 정보를 이용하여 현재 데이터를 처리함.

즉, 은닉층의 출력이 다시 은닉층의 입력으로 사용됨.

---



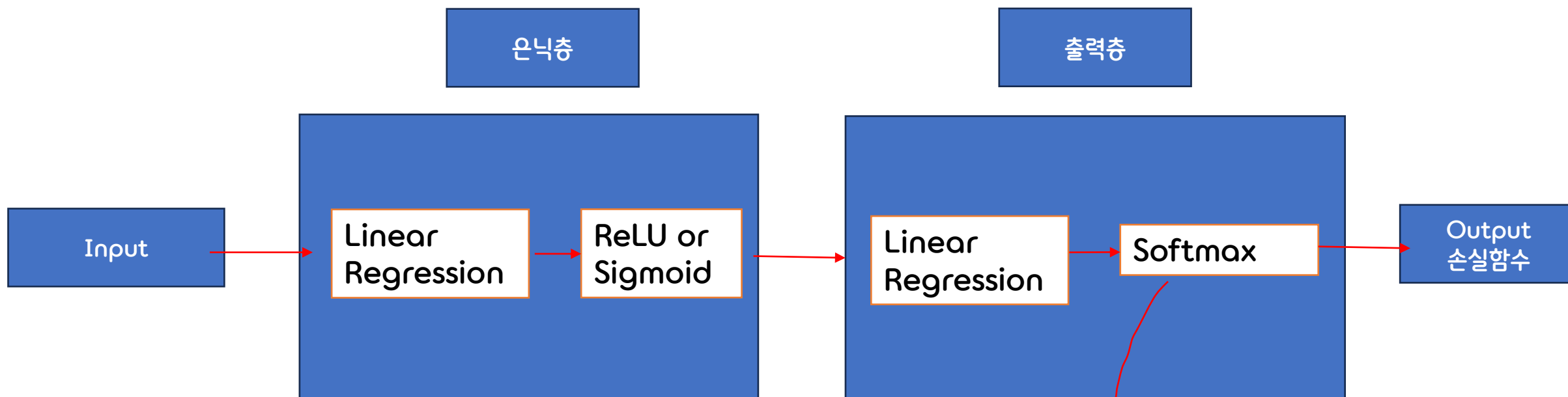
기존의 인공지능망



RNN

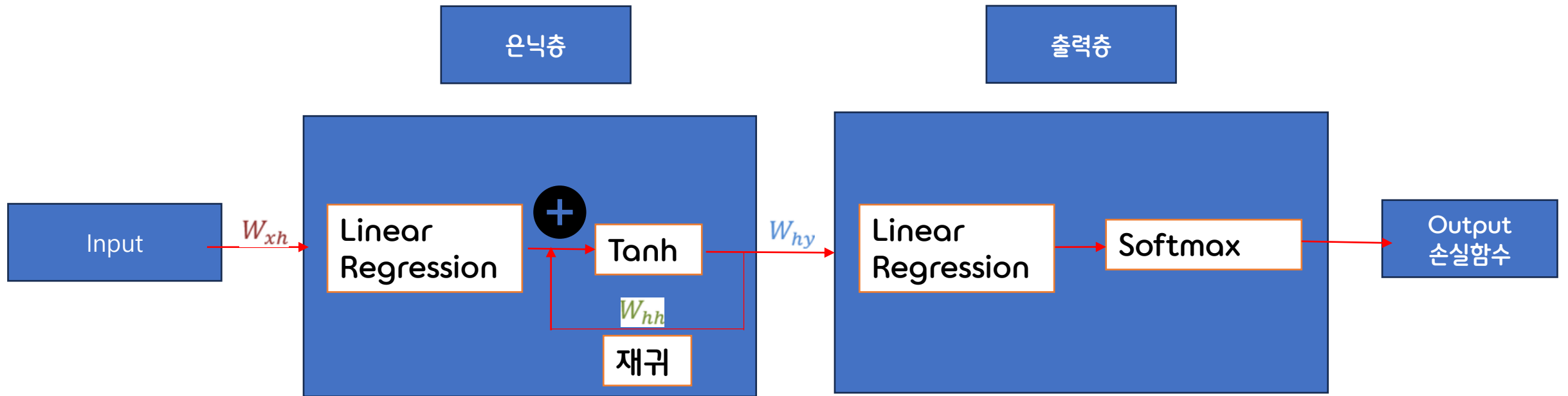
---

## 기존의 인공지능망

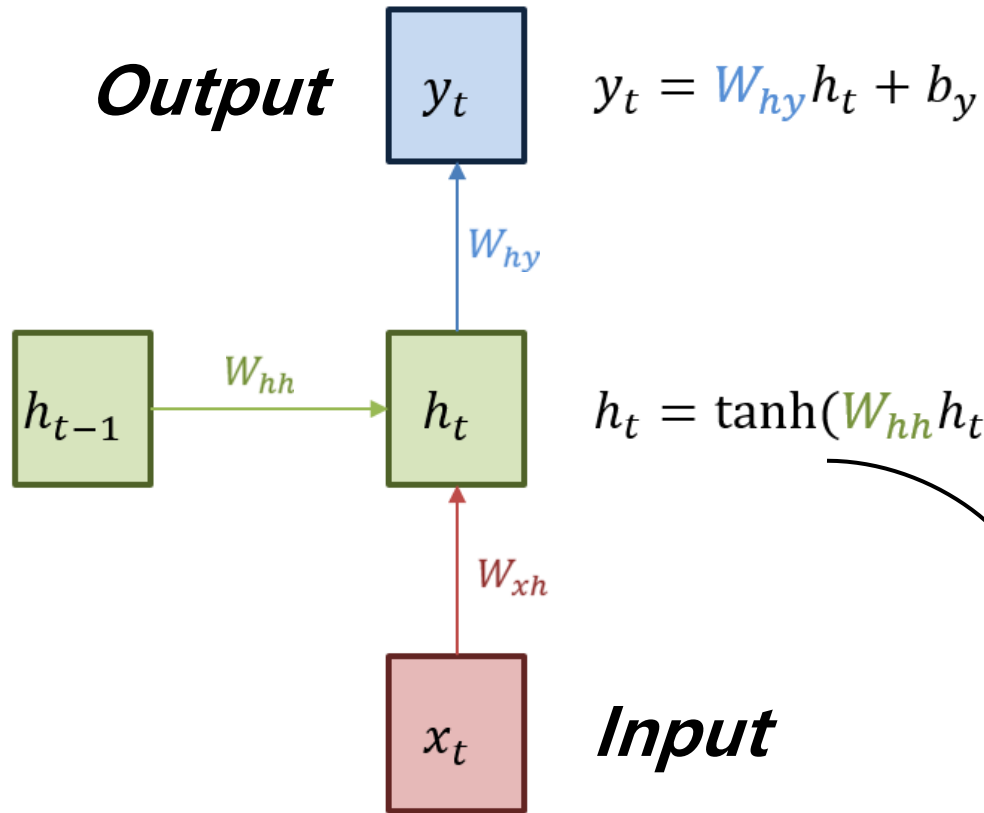


임의의 실수값으로 구성된 벡터 값을 받아 이를 확률분포로 변환 해줌.(0~1)

# RNN

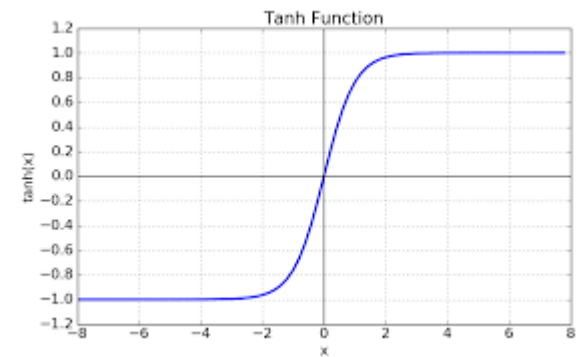


# RNN의 기본 구조



*RNN은 Hidden state를 구하는  
게 핵심!*  
: 현재시점의 데이터 + 이전 시점의 데이터

활성함수로  
Tanh 사용





RNN은 기본적으로 과거의 정보를 ‘기억’  
하면서 새로운 정보를 처리하는 것입니다

# 1986년 ~ 역전파 알고리즘의 선구자들 ~



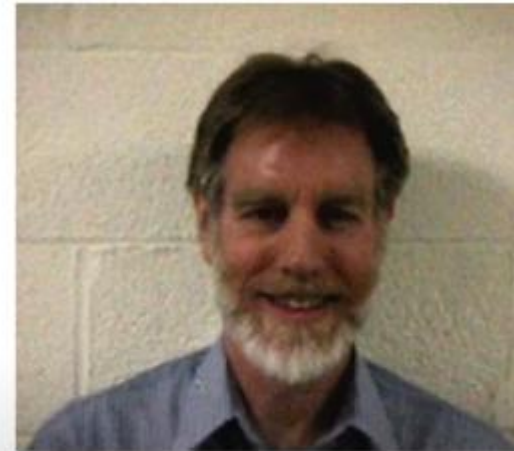
<https://www.psychologicalscience.org/observer/david-rumelhart>

David Rumelhart  
수리심리학자



<https://www.frontiersofknowledgeawards-fbbva.es/galardonados/geoffrey-hinton-2/>

Geoffrey Hinton  
실험심리학자  
컴퓨터과학자



<https://www.ccs.neu.edu/home/rjw/>

Ronald J. Williams  
컴퓨터과학자

# 1. 최초논문, 1985

- 오차역전파:인공신경망에서 출력과 실제 값 사이의 오차를 최소화하기 위해 가중치를 조정하는 방법
- 과정: 순전파-오차계산-오차역전파
- RNN에서 효과적 학습

## Learning representations by back-propagating errors

David E. Rumelhart\*, Geoffrey E. Hinton†  
& Ronald J. Williams\*

\* Institute for Cognitive Science, C-015, University of California,  
San Diego, La Jolla, California 92093, USA

† Department of Computer Science, Carnegie-Mellon University,  
Pittsburgh, Philadelphia 15213, USA

1986

We describe a new learning procedure, back-propagation, for networks of neurone-like units. The procedure repeatedly adjusts the weights of the connections in the network so as to minimize a measure of the difference between the actual output vector of the net and the desired output vector. As a result of the weight adjustments, internal 'hidden' units which are not part of the input or output come to represent important features of the task domain, and the regularities in the task are captured by the interactions of these units. The ability to create useful new features distinguishes back-propagation from earlier, simpler methods such as the perceptron-convergence procedure<sup>1</sup>.

There have been many attempts to design self-organizing neural networks. The aim is to find a powerful synaptic modification rule that will allow an arbitrarily connected neural network to develop an internal structure that is appropriate for a particular task domain. The task is specified by giving the desired state vector of the output units for each state vector of the input units. If the input units are directly connected to the output units it is relatively easy to find learning rules that iteratively adjust the relative strengths of the connections so as to progressively reduce the difference between the actual and desired output vectors<sup>2</sup>. Learning becomes more interesting but

more difficult when we introduce hidden units whose actual or desired states are not specified by the task. (In perceptrons there are 'feature analysers' between the input and output that are not true hidden units because their input connections are fixed by hand, so their states are completely determined by the input vector: they do not learn representations.) The learning procedure must decide under what circumstances the hidden units should be active in order to help achieve the desired input-output behaviour. This amounts to deciding what these units should represent. We demonstrate that a general purpose and relatively simple procedure is powerful enough to construct appropriate internal representations.

The simplest form of the learning procedure is for layered networks which have a layer of input units at the bottom; any number of intermediate layers; and a layer of output units at the top. Connections within a layer or from higher to lower layers are forbidden, but connections can skip intermediate layers. An input vector is presented to the network by setting the states of the input units. Then the states of the units in each layer are determined by applying equations (1) and (2) to the connections coming from lower layers. All units within a layer have their states set in parallel, but different layers have their states set sequentially, starting at the bottom and working upwards until the states of the output units are determined.

The total input,  $x_j$ , to unit  $j$  is a linear function of the outputs,  $y_i$ , of the units that are connected to  $j$  and of the weights,  $w_{ji}$ , on these connections

$$x_j = \sum_i y_i w_{ji} \quad (1)$$

Units can be given biases by introducing an extra input to each unit which always has a value of 1. The weight on this extra input is called the bias and is equivalent to a threshold of the opposite sign. It can be treated just like the other weights.

A unit has a real-valued output,  $y_j$ , which is a non-linear function of its total input

$$y_j = \frac{1}{1 + e^{-x_j}} \quad (2)$$

† To whom correspondence should be addressed.



## Geoffrey Hinton



Emeritus Prof. Computer Science, [University of Toronto](#)

Verified email at cs.toronto.edu - [Homepage](#)

[machine learning](#) [psychology](#) [artificial intelligence](#) [cognitive science](#) [computer science](#)

| TITLE                                                                                                                                                                                                                 | CITED BY | YEAR |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------|------|
| <a href="#">Imagenet classification with deep convolutional neural networks</a><br>A Krizhevsky, I Sutskever, GE Hinton<br>Advances in neural information processing systems 25                                       | 157893 * | 2012 |
| <a href="#">Deep learning</a><br>Y LeCun, Y Bengio, G Hinton<br>Nature 521 (7553), 436-44                                                                                                                             | 80600    | 2015 |
| <a href="#">Learning internal representations by error-propagation</a><br>DE Rumelhart, GE Hinton, RJ Williams<br>Parallel Distributed Processing: Explorations in the Microstructure of ...                          | 54315 *  | 1986 |
| <a href="#">Dropout: a simple way to prevent neural networks from overfitting</a><br>N Srivastava, G Hinton, A Krizhevsky, I Sutskever, R Salakhutdinov<br>The journal of machine learning research 15 (1), 1929-1958 | 51002    | 2014 |

# 논문 개요

- 신경망의 새로운 학습 절차인 역전파 설명
- 네트워크의 연결 가중치를 반복적으로 조정하여 실제 출력 벡터와 원하는 출력 벡터 간의 차이를 최소화
- BPTT(Back-Propagation Through Time)는 RNN의 학습에 필수적인 알고리즘
- BPTT는 시간축을 따라 오차를 전파하여, 긴 시퀀스 데이터에서도 효과적으로 학습할 수 있도록 함

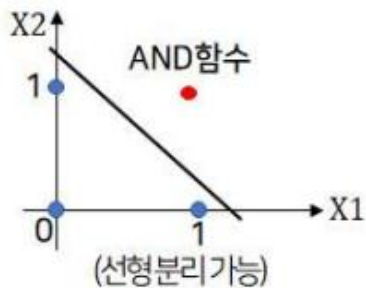
"역전파 알고리즘의 핵심  
은 편미분과 체인룰이다"

본 논문은 현대에 활용되는 신경  
망 학습법의 길을 열었다.

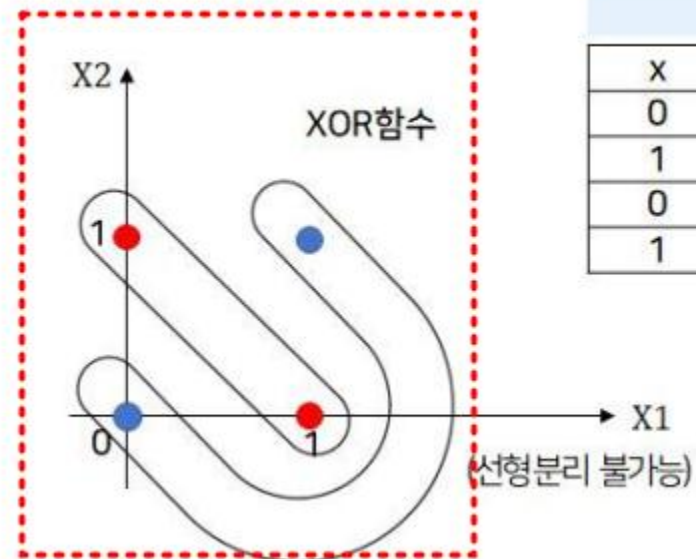
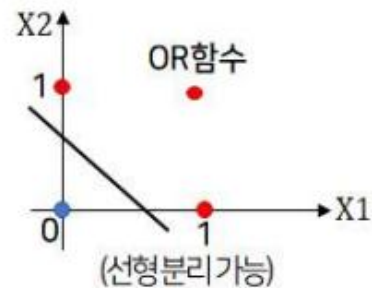
# (논문 발표 이전) 학습 알고리즘의 부재

- 가중치 업데이트 어려움: 다층신경망에서 각 층의 가중치를 효율적으로 업데이트 하는 방법이 필요했지만, 효과적인 알고리즘이 없었음
- 선형 분리 불가능 문제를 해결할 수 없었음

| AND함수 |   |    |
|-------|---|----|
| x     | y | 결과 |
| 0     | 0 | 0  |
| 1     | 0 | 0  |
| 0     | 1 | 0  |
| 1     | 1 | 1  |



| OR함수 |   |    |
|------|---|----|
| x    | y | 결과 |
| 0    | 0 | 0  |
| 1    | 0 | 1  |
| 0    | 1 | 1  |
| 1    | 1 | 1  |



| XOR함수 |   |    |
|-------|---|----|
| x     | y | 결과 |
| 0     | 0 | 0  |
| 1     | 0 | 1  |
| 0     | 1 | 1  |
| 1     | 1 | 0  |



# (논문 출판 이후) 역전파 알고리즘 도입

- 신경망의 부활 : 역전파를 통해 다양한 문제를 해결
- 다양한 응용을 통한 신경망 발전
  - ✓ 이미지 인식
  - ✓ 음성 인식
  - ✓ 자연어 처리
- 딥러닝의 기초:
  - ✓ 모델 발전에 큰 기여
  - ✓ 깊은 신경망 구조와 대규모 데이터 셋 활용 학습 가능해짐
  - ✓ 다양한 아키텍처 개발: CNN, RNN, LSTM 등

# Q1: RNN은 왜 '순환(Recurrent)' 신경망이라고 불리나요?

## ✓ A1: '순환'이란 무엇일까요?

'순환'이란, 무언가가 다시 제자리로 돌아오는 것을 말해요.

예를 들어, 시계의 초침이 뱅~ 돌아서 다시 12를 지나가듯이, 계속 반복되고 이어지는 흐름이죠.

## 💡 RNN은 뭘 순환할까요?

RNN(Recurrent Neural Network)은 과거의 정보를 기억해서, 그것을 다시 사용해요.

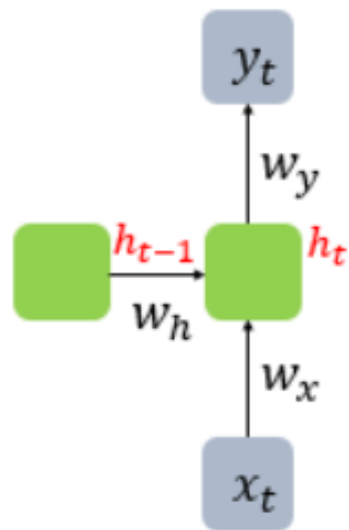
- 우리가 이야기할 때, **지금 무슨 말을 할지 결정할 때** 예전에 무슨 말을 했는지가 중요하잖아요?
  - 예: "나는 밥을 **먹고** 나서... 산책을 했다."
  - 여기서 '먹고'라는 단어가 나와야만 그다음 문장이 자연스럽죠.

RNN도 똑같아요!

→ 과거에 배운 내용을, 다음에 또 사용할 수 있도록 기억을 되살려서 순환시켜요.

# 인코더 디코더 아키텍처 기반 RNN(Seq2Seq 모델)

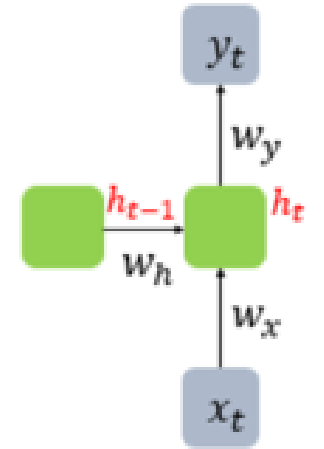
- $x_t$ : 현재 시점의 입력 (예: 단어 벡터)
- $h_{t-1}$ : 이전 시점의 hidden state
- $h_t$ : 현재 시점의 hidden state (출력과 다음 단계의 입력으로 사용)
- $y_t$ : 현재 시점의 최종 출력 (예: 예측 단어)
- $W_x, W_h, W_y$ : 각 연결에 대한 가중치



# # 문장 번역

## 예시

- 입력 문장 (영어): "I love you"
- 출력 문장 (프랑스어): "Je t'aime"



| 프랑스어 | 의미          | 한글 독음 | 발음 팁                |
|------|-------------|-------|---------------------|
| Je   | 나 (I)       | 쥬     | 영어 "zh" + "uh" 느낌   |
| t'   | 너를 (you)    | 뚜의 축약 | 다음 단어와 연결됨          |
| aime | 사랑한다 (love) | 엠(땜)  | 비음화되어 "엠"보다는 "땜" 느낌 |

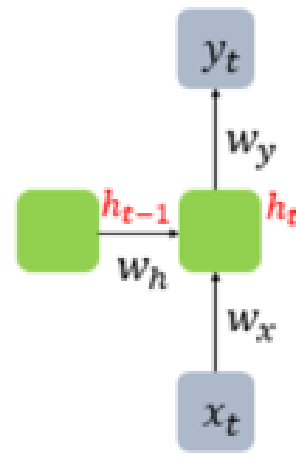
## ● < 인코더 구성 요소 >

### 1 $x_t$ (엑스 티)

- 뜻: 시점  $t$ 에서 들어온 단어
- 예: 영어 문장 "I love you"에서  $x_1 = \text{"I"}$ ,  $x_2 = \text{"love"}$ ,  $x_3 = \text{"you"}$

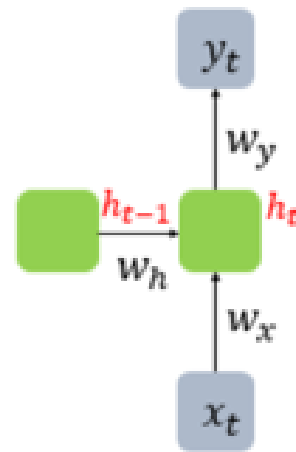
### 2 $h_{t-1}$ (에이치 티 마이너스 원)

- 뜻: 이전까지 들은 내용을 요약한 기억
- 예:  $x_1 = \text{"I"}$ 가 들어왔을 때는 초기 기억값(보통 0),  
 $x_2 = \text{"love"}$ 가 들어왔을 땐  $h_1 = \text{"I"}$



### 3 $h_t$ (에이치 티)

- 뜻: 지금 단어까지 포함한 새로운 기억 (업데이트된 상태)
- 예:  $x_3 = \text{"you"}$  가 들어오면,  
 $h_3 = \text{"I love you"}$  처럼 문장을 압축해서 담음

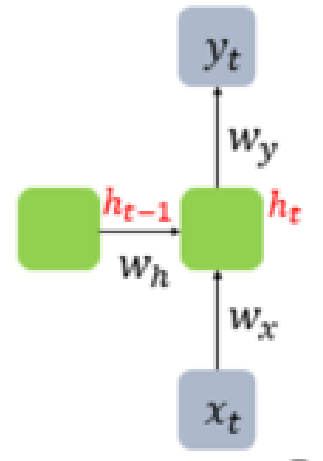


### 4 $y_t$ (와이 티)

! 인코더에서는 보통 쓰지 않음 (디코더가 주로 출력함)

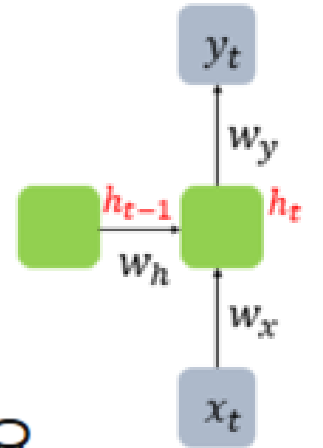
- 뜻: 시점  $t$ 에서의 출력값
- 예: 디코더에서는 번역 결과 단어가 출력됨  
예:  $y_1 = \text{"Je"}$  (프랑스어)
- 쉬운 말: "이제 말할 차례일 때 나오는 단어"  
→ 인코더에서는  $y_t$ 를 출력하지 않기도 해요!

# # (정리) RNN 인코더 부분



1. RNN은 문장을 한 단어씩 차례로 읽으며 처리해요.
2. 각 단어는 입력  $x_t$ 로 표현돼요. 예: "I", "love", "you".
3. 처음에는 아무 기억도 없어서 기억값  $h_0$ 은 0이에요.
4. 첫 단어 "I"를 보면, 그걸 기억해서  $h_1$ 에 저장해요.
5. 다음 단어 "love"가 들어오면, "I"의 기억과 합쳐서 새 기억  $h_2$ 를 만드어요.

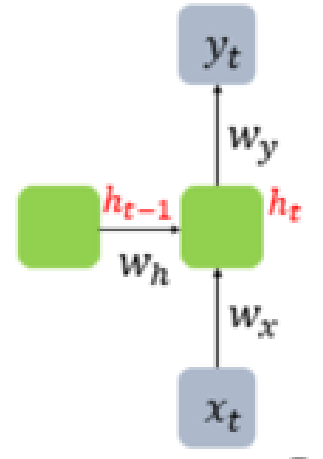
## # (정리) RNN 인코더 부분



6. 또 다음 단어 "you"가 들어오면, "I love" 기억을 바탕으로  $h_3$ 가 돼요.
7. 이렇게 RNN은 이전 기억  $h_{t-1}$ 과 새 입력  $x_t$ 를 같이 사용해요.
8. 단어 하나 볼 때마다 기억을 새로 갱신하는 게 핵심이에요.
9. 그림에서처럼 네모 박스 안에서 여러 가중치  $w_x, w_h, w_y$ 로 계산이 일어나요.
10. 마지막 기억  $h_t$ 는 문장의 요약처럼 다음 번역 등에 쓰여요.

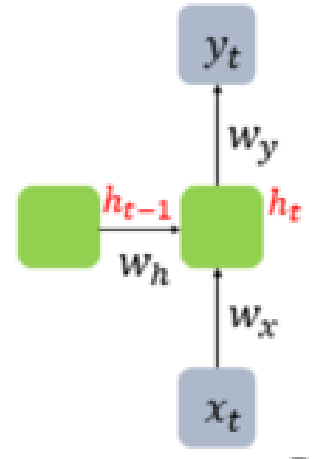


## # (정리하면) RNN 디코더 부분

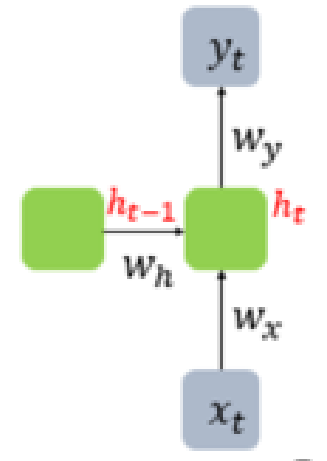


1. 디코더는 인코더가 만든 마지막 기억  $h_t$ 를 받아서 시작해요.
2. 이 기억은 "전체 문장을 압축한 정보"예요.
3. 디코더는 이 정보를 바탕으로 한 단어씩 문장을 생성해요.
4. 처음에는 특별한 시작기호(예: <start>)를 입력으로 넣어요.
5. 이 시작기호와 인코더 기억  $h_t$ 를 이용해 첫 단어를 예측해요.

## # (정리하면) RNN 디코더 부분



6. 예측된 첫 단어는 다음 입력으로 사용돼요.
7. 예: "I love you" 번역이면, 첫 출력이 "나"일 수 있어요.
8. 그다음엔 "나"를 입력으로 넣고 다시 단어를 생성해요.
9. 이런 방식으로 디코더는 번역, 요약, 대답 등을 완성해요.
10. 마지막엔 <end> 같은 종료 기호가 나오면 멈춰요.



RNN은 '이전까지의 모든 입력 시퀀스에 대한 정보를 압축하여 히든 스테이트에 담아 전달한다.'

- RNN은 한계가 발생한다.
- 장기 의존성 문제 (Long-term dependency problem)이다. 일반적인 RNN은 시퀀스가 길어질수록 오래 전의 정보를 기억하는 데 어려움을 겪는다.
- 이를 "소실 기울기(vanishing gradient)" 문제라고 하는데,
- 역전파 과정에서 기울기 값이 너무 작아져서 과거의 정보가 현재의 가중치 업데이트에 거의 영향을 미치지 못하게 되는 현상이다.
- 이로 인해 RNN은 먼 과거의 중요한 정보를 잊어버리거나 제대로 학습하지 못하는 경향이 있다.

이러한 한계를 극복하기 위해

- LSTM(Long Short-Term Memory)이나 GRU(Gated Recurrent Unit)\*\*와 같은 RNN의 변형들이 개발되었다.
- 이들은 게이팅 메커니즘을 통해 어떤 정보를 기억하고 어떤 정보를 잊을지 선택적으로 제어함으로써 장기 의존성 문제를 완화하고 더 긴 시퀀스의 정보를 효과적으로 기억할 수 있도록 한다.

# LSTM(Long Short-Term Memory) 신경망

셀 + 입력게이트 + 출력게이트 + 망각게이트

# 논문 정보

- 제목: Long Short-Term Memory
- 저널: Neural Computation
- 권(호): 9(8)
- 페이지: 1735-1780
- 발행년도: 1997년

## LONG SHORT-TERM MEMORY

NEURAL COMPUTATION 9(8):1735-1780, 1997

Sepp Hochreiter

Fakultät für Informatik

Technische Universität München

80290 München, Germany

hochreit@informatik.tu-muenchen.de

<http://www7.informatik.tu-muenchen.de/~hochreit>

Jürgen Schmidhuber

IDSIA

Corso Elvezia 36

6900 Lugano, Switzerland

juergen@idsia.ch

<http://www.idsia.ch/~juergen>

### Abstract

Learning to store information over extended time intervals via recurrent backpropagation takes a very long time, mostly due to insufficient, decaying error back flow. We briefly review Hochreiter's 1991 analysis of this problem, then address it by introducing a novel, efficient, gradient-based method called "Long Short-Term Memory" (LSTM). Truncating the gradient where this does not do harm, LSTM can learn to bridge minimal time lags in excess of 1000 discrete time steps by enforcing *constant* error flow through "constant error carousels" within special units. Multiplicative gate units learn to open and close access to the constant error flow. LSTM is local in space and time; its computational complexity per time step and weight is  $O(1)$ . Our experiments with artificial data involve local, distributed, real-valued, and noisy pattern representations. In comparisons with RTRL, BPTT, Recurrent Cascade-Correlation, Elman nets, and Neural Sequence Chunking, LSTM leads to many more successful runs, and learns much faster. LSTM also solves complex, artificial long time lag tasks that have never been solved by previous recurrent network algorithms.

# Sepp Hochreiter(젍 호크라이터)

- 소속: Technische Universität München (독일 뮌헨 공과대학교)
- 학부: Fakultät für Informatik (정보학부)
- 주소: 80290 München, Germany (독일 뮌헨)
- 이메일: [hochreit@informatik.tu-muenchen.de](mailto:hochreit@informatik.tu-muenchen.de)
- 웹사이트: <http://www7.informatik.tu-muenchen.de/~hochreit>

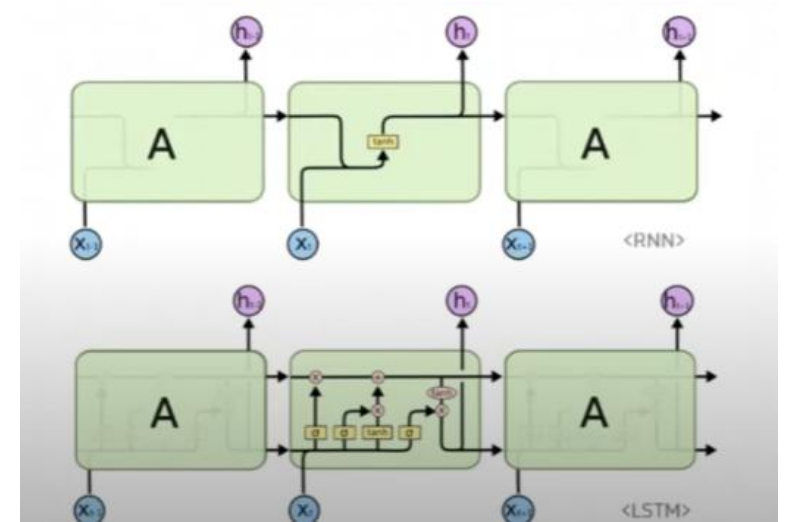


# Jürgen Schmidhuber( 위르겐 슈미드후버)

- 소속: IDSIA (Dalle Molle Institute for Artificial Intelligence Research)
- 주소: Corso Elvezia 36, 6900 Lugano, Switzerland (스위스 루가노)
- 이메일: [juergen@idsia.ch](mailto:juergen@idsia.ch)
- 웹사이트: <http://www.idsia.ch/~juergen>

# 기억하는 장치

- (긴문장에서)중요한 정보는 기억하자
- 필요없는 정보는 버리자.



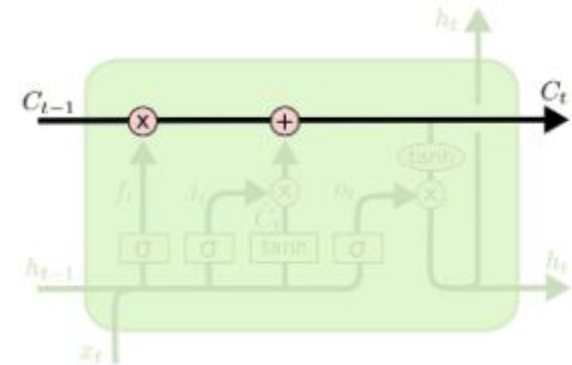
“Cell state”

“정보를 없애거나 더할 수 있는 능력이 있다”

# Cell State

1. 망각 게이트(Forget Gate)
  - 중요성 판단+중요하지 않으면 제거
2. 입력 게이트(Input Gate)
  - (새로운 정보가 들어왔을때)중요성 판단+중요하면 셀상태 정보 추가+장기기억에 기여
3. 출력 게이트(Out Gate)
  - 정보 분석+다음레이어 전달 또는 다음 시간

“컨베이너 벨트와 같다”



LSTM의 cell state

입력 게이트와 망각 게이트는 동시에 계산되지만,

- 셀 상태 업데이트 적용 순서는 망각 게이트 그리고 입력게이트이다.
- 즉, 논리적 판단은 동시에 일어나고,
- 메모리 업데이트는 '먼저 지우고 -> 나중에 채우는 구조'
- 예를 들면, 빨래 바구니를 비유로 들면,
  - ✓1단계: 더 이상 입지 않을 옷(망각)을 골라내고 →
  - ✓2단계: 새 옷(입력)을 바구니에 넣는 순서입니다.

# '망각정보'의 중요성 판단 시그모이드 함수

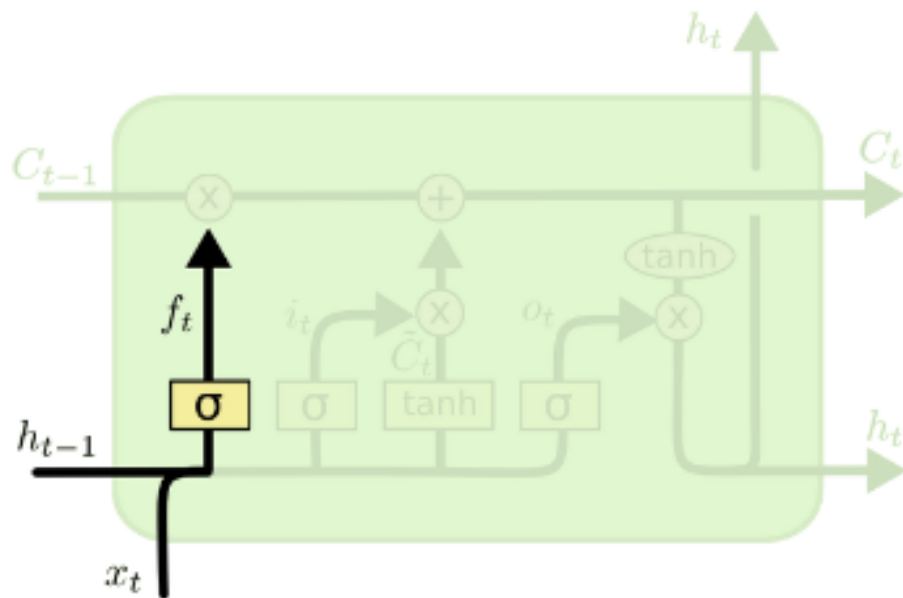
- 셀 상태의 값 0~1 사이의 값 출력
- 0에 가까울 수록 : 잊어야 할 정보
- 1에 가까우면 : 기억해야할 정보

# Forget gate layer

“시그모이드”

“1이면 보존”

“0이면 잊어라”

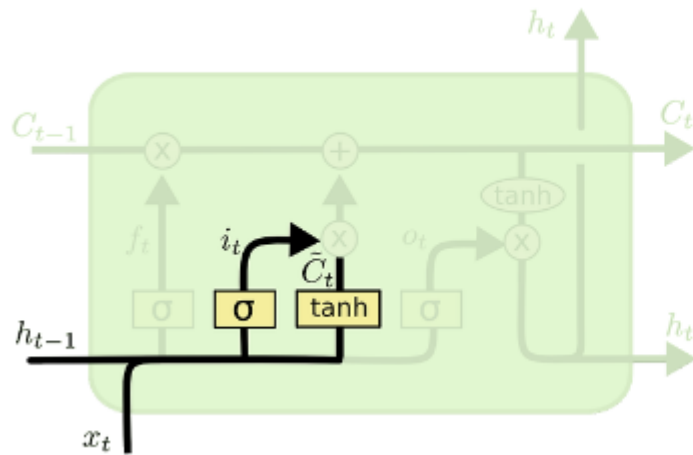


$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

“어떤 새로운 정보를 Cell state에 저장할까?”

# Input gate layer

“새로운 정보의 중요성 판단 메커니즘”  
시그모이드함수 X 하이퍼볼릭 탄젠트 함수



$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

# Why? 시그모이드x 탄젠트 함수

- 문장에서 '하지만', '그러나' >> 감정이 빠질 수 있다.
- LSTM 이전에 수집한 정보 수정의 필요성 있다.
- 시그모이드 함수값 0.9 출력했다면 '완전 긍정감정일 것'
- 하이퍼볼릭 탄젠트 함수 -0.8 >> 감정 전환 시도
- 두개의 곱: -0.72

"긍정적인 감정 정보를 완화 + 문장의 감정톤을 부정적으로 전환하는데 기여함"



<예시> “그 영화 정말 재미있었어. **하지만** 결말이 너무 별로였어.”

## 1. 앞부분:

- “그 영화 정말 재미있었어.” → **감정 분석 결과: 긍정 (+0.9)**
- 이 정보는 LSTM 셀 상태에 저장됩니다.

## 2. 뒷부분:

- “하지만 결말이 너무 별로였어.” → **부정적인 표현 등장**
- 특히 ‘하지만’이라는 전환 표현이 나타나면서 **앞에 있던 긍정 감정을 덮을 필요가 생김.**

## ○ 시그모이드 함수 ( $\sigma$ )

- 입력: "결말이 너무 별로였어"
- 출력값: **0.9**
- 의미:  
→ "이 감정 정보는 **중요**하니 셀 상태에 **반영**하자."

## ○ 곱셈 결과:

- 계산:  $0.9 \times (-0.8) = -0.72$
- 의미:  
→ "**중요한 부정** 감정이 셀 상태에 **강하게 반영**됨"

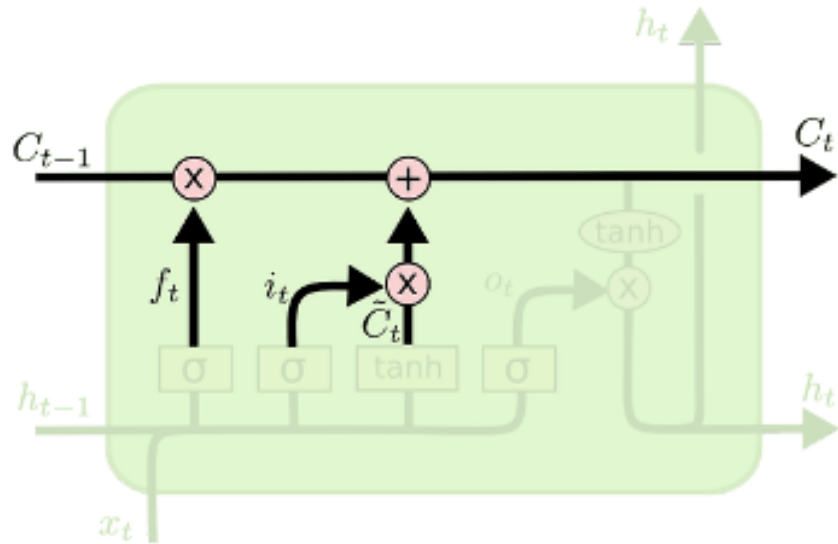
## ○ 탄젠트 함수 ( $\tanh$ )

- 입력 감정: 부정적인 표현
- 출력값: **-0.8**
- 의미:  
→ "이건 **부정적인** 감정이다."

**"시그모이드 × 탄젠트 = 중요도 × 감정 방향"**

**→ 셀 상태에 어떤 감정을 얼마나 반영할지 결정한다!"**

# Cell State 업데이트

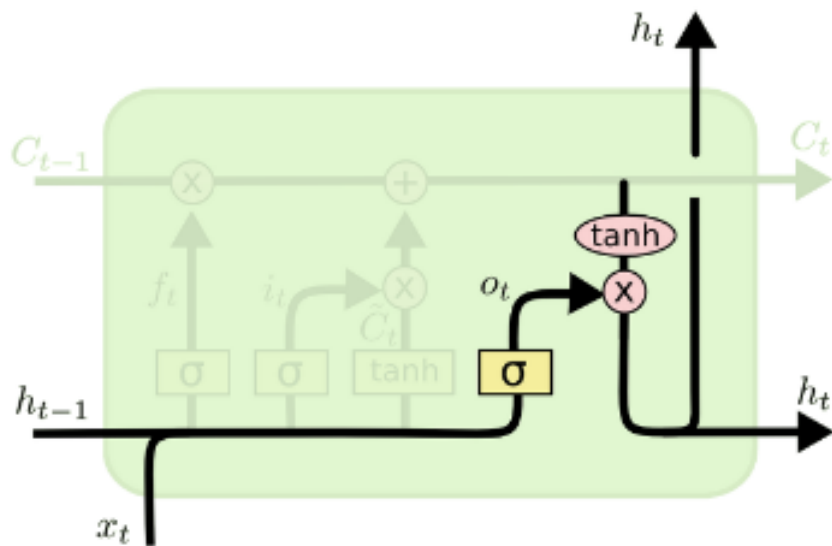


$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

“버릴 건 버리고 중요한 새정보만 추가해서,  
지금 시점의 새로운 기억 완성 ”

# Output gate layer

현재의 셀 상태를 바탕으로 최종적으로 출력할 정보 정하기



$$o_t = \sigma(W_o [h_{t-1}, x_t] + b_o)$$

$$h_t = o_t * \tanh(C_t)$$

# LSTM 셀 상태 업데이트 (5단계)

## ◆ 1단계: 이전 기억 받아오기

- 입력: 이전 셀 상태  $C_{t-1}$ , 이전 은닉 상태  $h_{t-1}$
- 역할: 지금 기억을 만들기 위한 출발점

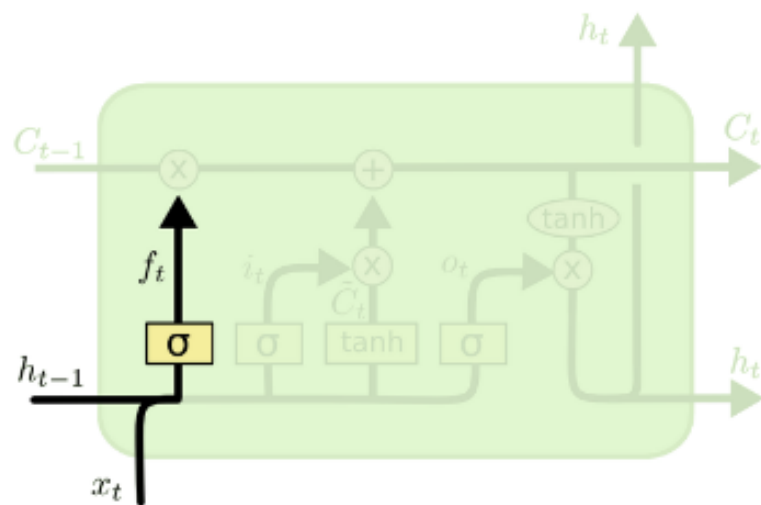
## ◆ 2단계: 망각 게이트 계산 (Forget Gate)

- 수식:  $f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$
- 기능: 이전 기억 중 버릴 정보를 선택
- 결과:  $f_t \cdot C_{t-1}$

“시그모이드”

“1이면 보존”

“0이면 잊어라”



$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$

### ◆ 3단계: 입력 게이트 계산 (Input Gate)

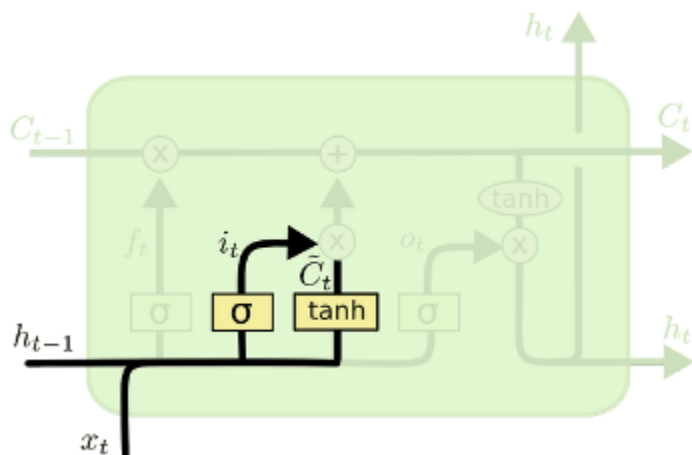
- 수식:

- $i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$
- $\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$

- 기능: 새로운 입력 중 **추가할 정보**를 선택
- 결과:  $i_t \cdot \tilde{C}_t$

“어떤 새로운 정보를 Cell state에 저장할까?”

“새로운 정보의 중요성 판단 메커니즘”  
시그모이드함수 X 하이퍼볼릭 탄젠트 함수



$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

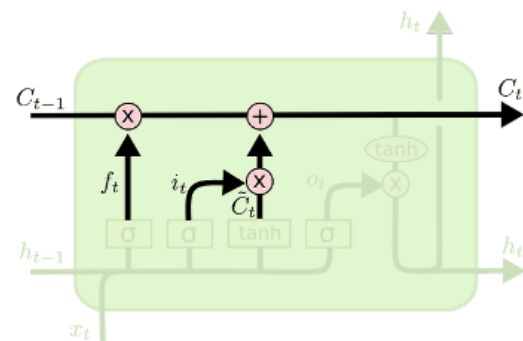


## ◆ 4단계: 셀 상태 업데이트

- 수식:

$$C_t = f_t \cdot C_{t-1} + i_t \cdot \tilde{C}_t$$

- 기능:
  - 버릴 건 버리고,
  - 중요한 새 정보만 추가해서  
→ 지금 시점의 새로운 기억 완성



$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

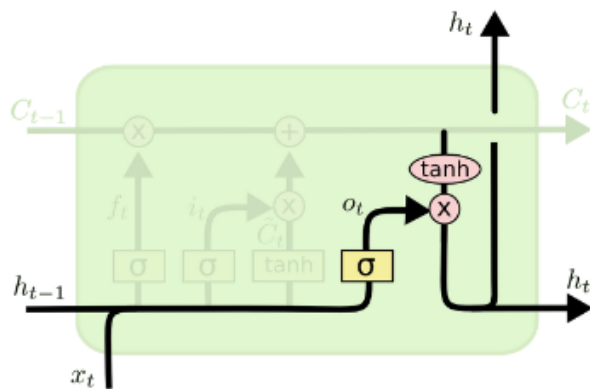
## ◆ 5단계: 출력 게이트 → 다음 단계로 전달

- 수식:

- $o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$

- $h_t = o_t \cdot \tanh(C_t)$

- 기능: 셀 상태에서 꺼낼 정보를 은닉 상태로 만들어 다음 시점에 넘김



$$o_t = \sigma(W_o [h_{t-1}, x_t] + b_o)$$

$$h_t = o_t * \tanh(C_t)$$